

Introducao

palha sobre o que sao particle systems, para que sao usados, que sao computacionalmente pesados, etc. quisemos usar algoritmos minimamente baseados em fisicas e colicoes

Algoritmos de integracao

Para poder simular os movimentos das particulas, precisamos de integrar o mesmo, ou seja, usar passos discretos para simular um movimento que realisticamente seria continuo, baseando-nos nas equacoes de movimento.

O grupo investigou algumas alternativas, tais como:

temos de explicar o que eles sao??

- Basic Verlet Integration
- Velocity Verlet
- Euler integration
- Leapfrog Integration
- Position Verlet
- Runge-Kutta Methods (e.g., RK4)
- Backward Euler

Tinhamos como prioridade obter boa performance mas tambem estabilidade razoavel da simulacao, pelo que optamos por usar Basic Verlet, que nos pareceu um bom compromisso, dadas as comparacoes efetuadas por diversas fontes???????

Collisoes

Determinismo

Ao usar um Δt fixo, a simulacao usando Basic Verlet passa a ser deterministica. Esta propriedade pareceu-nos extremamente interessante, visto que o mesmo conjunto de dados de input ira sempre gerar o mesmo output. Assim, por exemplo, poderiamos gerar imagens dando cores as particulas: como elas iriam sempre calhar nos mesmos sitios, as cores iniciais poderiam ser previstas de modo a representarem as cores das imagens.

Otimizacoes

O algoritmo atual percorre N^2 particulas para detetar colisoes, comparando todas as particulas com todas as outras particulas. Isto e muito inefficiente, e podera ser melhorado tirando proveito do facto de que colisoes entre particulas distantes nao tem de ser computadas pois nao irao surtir qualquer efeito. Para alem disso, nao e paralelizavel, deixando de lado possiveis ganhos de performance.

Assim, implementamos algumas otimizacoes:

Binning

Para aproveitar o facto de colisões so surtirem efeito se ocorrerem entre particulas proximas, podemos usar um algoritmo de binning, dividindo o espaco numa grelha (em bins) e colidindo apenas particulas entre celulas adjacentes.

Assim, no inicio de cada passo de simulacao, o algoritmo coloca cada particula numa celula da grelha com base na sua posicao. Para calcular colisões, basta, para cada celula, iterar as suas particulas, calculando colisoes apenas com as particulas dentro da propria celula e nas celulas que a rodeiam.

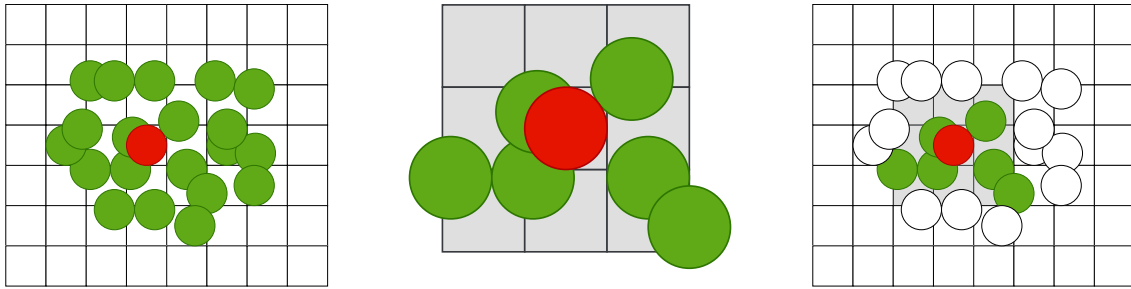


Figure 1: Partículas com que a partícula vermelha precisa de ser comparada.

Esquerda: sem binning. Meio: partículas em células adjacentes. Direita: com binning.

numeros a mostrar o quanto isto melhorou

Multithreading

Visto o espaço agora estar organizado numa grelha, e possível implementar multithreading neste algoritmo, atribuindo uma parte da grelha a cada thread.

No entanto, diferentes threads podem aceder a células adjacentes, provocando data races nas colisões entre as partículas das mesmas. Estas data races tornam o algoritmo não determinístico, o que vai contra o nosso objetivo.

Para voltar a tornar a simulação determinística, decidimos fazer 2 passes de simulação distintos, procurando evitar que threads diferentes possam estar a processar a mesma célula:

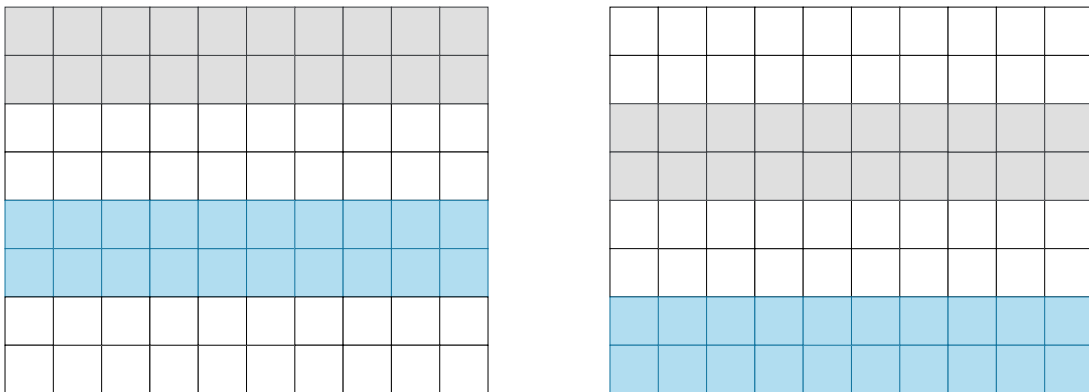


Figure 2: Os espaços cinzento e azul estão a ser processados por duas threads distintas. São feitos dois passes (esquerda e direita), garantindo que ao processar células azuis e cinzentas nunca há células adjacentes a serem processadas em simultâneo

Padding

Ao calcular quais as células que estão em redor de uma dada célula, existem várias posições na grelha que precisam de cuidados especiais, o que complica o código, como as células que se situam nas edges da grelha, e especialmente nos cantos. Seria necessário ter casos especiais em que determinávamos se se trata de uma destas situações, e impedir, por exemplo, a comparação com células acima devido a estas não existirem.

Assim, decidimos acrescentar uma camada de padding, ou seja, bins que nunca contêm qualquer partícula nem são processadas, de forma a uniformizar o código e remover condições desnecessárias do mesmo.

Esta otimização torna-se especialmente relevante para evitar thread divergence, que será útil para poder executar a deteção de colisões na GPU no futuro.

Compute shaders

Com o objetivo de aumentar ainda mais o número de partículas na nossa simulação, concluímos que apenas com a capacidade computacional de uma GPU seria possível.

Colisao basica

Numa primeira fase, decidimos implementar novamente o algoritmo ineficiente que compara todas as particulas com todas as outras particulas.

Apesar dos problemas obvios deste algoritmo, ainda assim tivemos enormes ganhos de performance

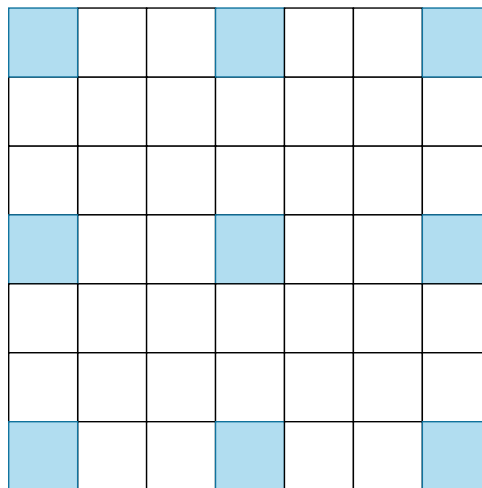
numeros a mostrar o quanto melhorou

Binning

A solucao ideal, usando binning, traz uma nova dimensao aos problemas encontrados na implementacao de multithreading. Tendo uma arquitetura paralela, evitar race conditions torna-se desafiante, mas necessario.

Infelizmente, para manter o determinismo, o passo de gerar as bins tera sempre de ser feito no cpu, ou pelo menos sorted no mesmo, para garantir que as particulas sao sempre processadas na mesma ordem, pelo que nao conseguimos evitar perdas vindas da latencia de transferencia de dados GPU↔CPU.

Nas colisoes, a solucao de usar varios passes diferentes, na solucao de multithreading, tera de ser adaptada a arquitetura da GPU: nao e razoavel que uma thread processe uma zona inteira do espaco da grelha, mas sim uma unica celula. Assim, para cada celula, teremos de garantir que existe um espaco de 2 celulas livre tanto na vertical como na horizontal:



Enquanto na solucao com multithreading seria suficiente 2 passes de simulacao, aqui serao necessarios 9 para que todas as celulas sejam devidamente processadas:

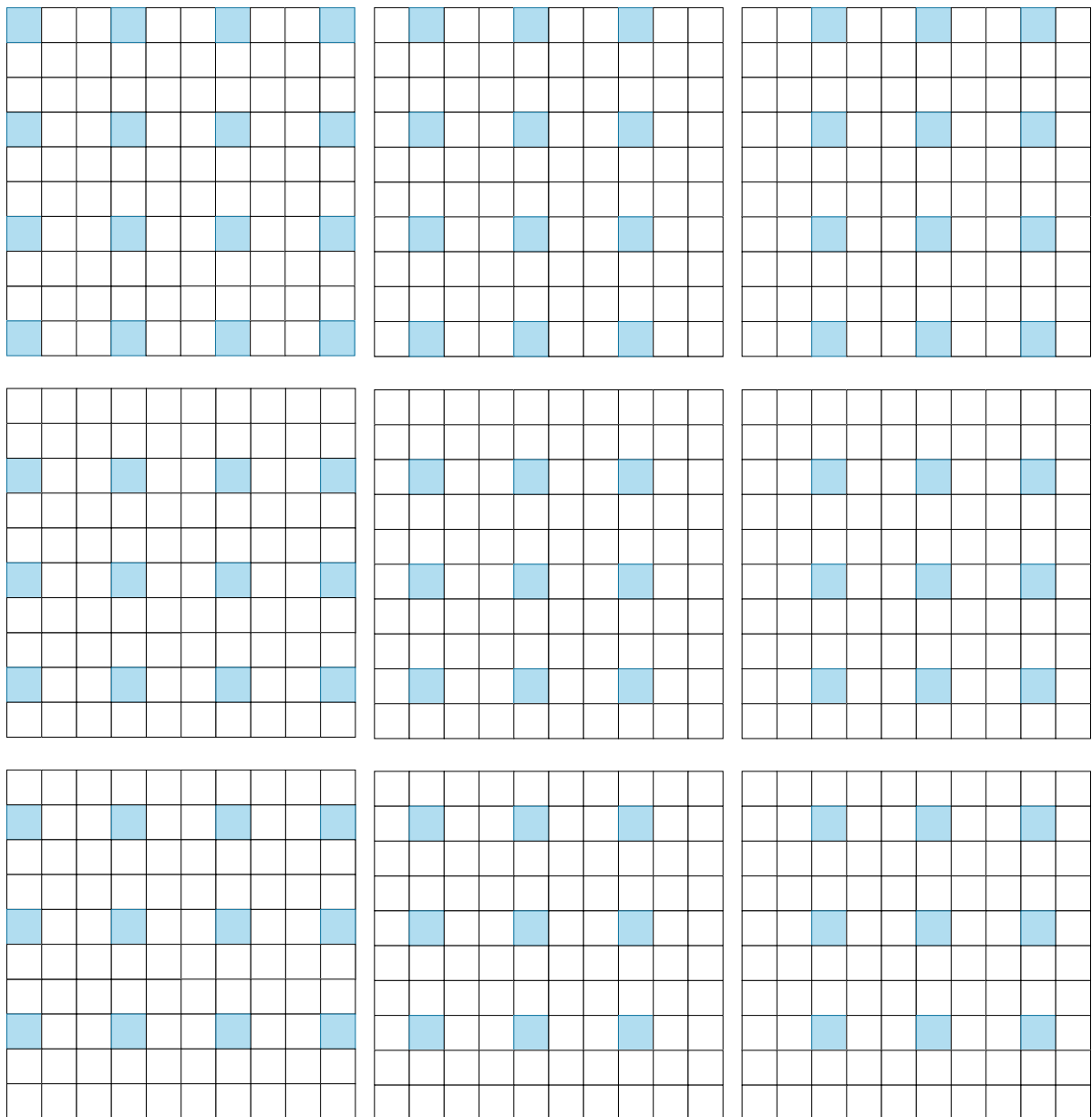


Figure 4: Os 9 passes necessarios para calcular as colisoes de todas as celulas, evitando data races

substeps

vert e frag shaders, explicar como os dados ja estao na gpu entao temos boa perf

como imagens sao carregadas

mostrar um resultado final, sq um video para o youtube

otimizacoes futuras: algoritmo do dudu, e como passamos a tirar partido do facto de existirem workgroups

reduce para contar o numero de particulas?